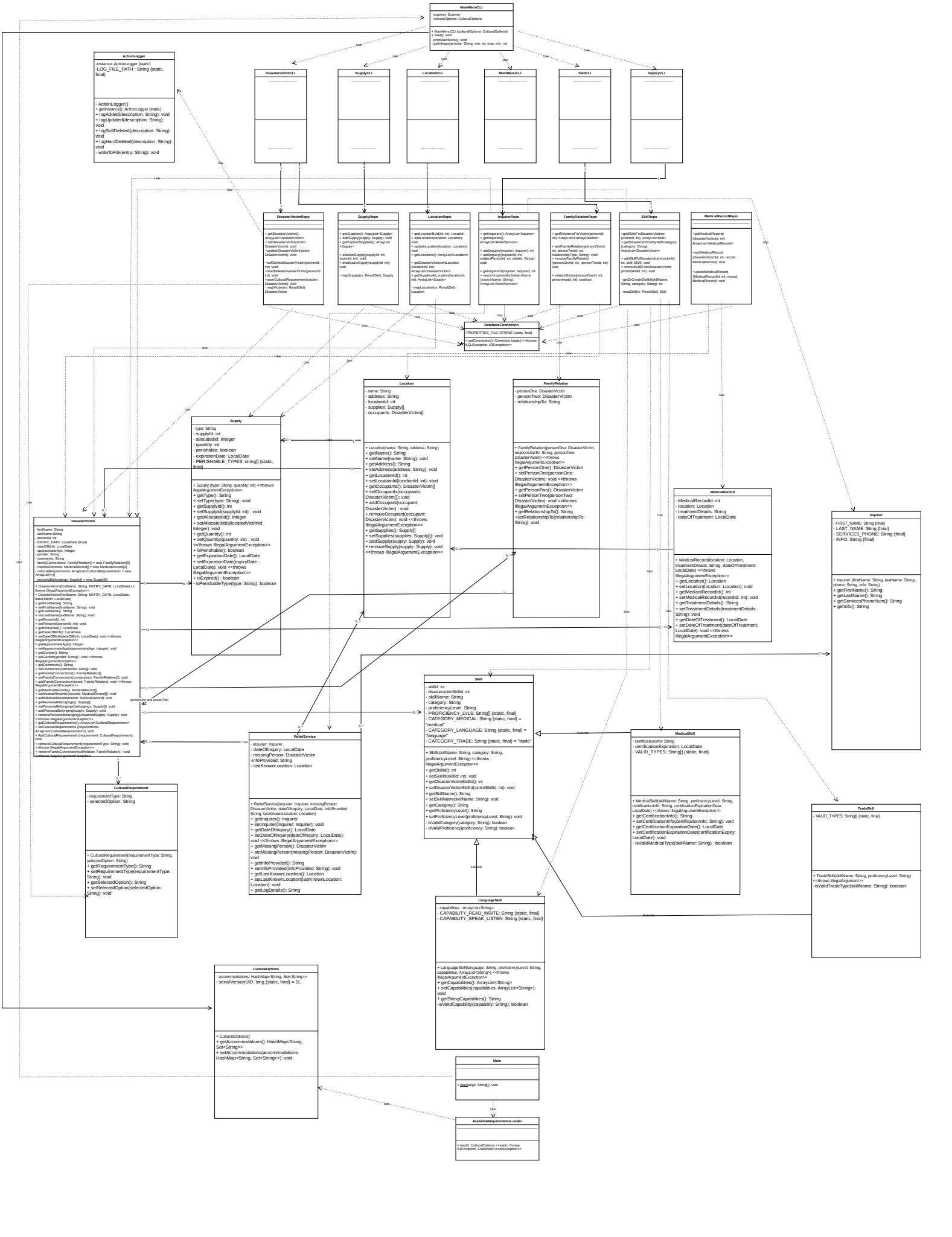

MainMenuCLI
- scanner: Scanner
- culturalOptions: CulturalOptions
+ MainMenuCLI (culturalOptions: CulturalOptions)
+ start(): void
- printMainMenu(): void
- getIntInput(prompt: String, min: int, max: int) : int
ActionLogger
-instance: ActionLogger (static)
-LOG_FILE_PATH : String (static, final)
- ActionLogger()
+ getInstance(): ActionLogger (static)
+ logAdded(description: String): void
+ logUpdated(description: String): void
+ logSoftDeleted(description: String): void
+ logHardDeleted(description: String): void
- writeToFile(entry: String): void
DisasterVictimCLI
SupplyCLI
LocationCLI
MainMenuCLI
SkillCLI
InquiryCLI
DisasterVictimRepo
SupplyRepo
LocationRepo
InquirerRepo
FamilyRelationRepo
SkillRepo
MedicalRecordRepo
DatabaseConnection
-PROPERTIES_FILE: STRING (static, final)
+ getConnection(): Connectio [static] <<throws SQLException, IOException>>
Location
- name: String
- address: String
- locationId: int
- supplies: Supply[]
- occupants: DisasterVictim[]
+ Location(name: String, address: String)
+ getName(): String
+ setName(name: String): void
+ getAddress(): String
+ setAddress(address: String): void
+ getLocationId(): int
+ setLocationId(locationId: int): void
+ getOccupants(): DisasterVictim[]
+ setOccupants(occupants: DisasterVictim[]): void
+ addOccupant(occupant: DisasterVictim) : void
+ removeOccupant(occupant: DisasterVictim): void <<throws IllegalArgumentException>>
+ getSupplies(): Supply[]
+ setSupplies(supplies: Supply[]): void
+ addSupply(supply: Supply): void
+ removeSupply(supply: Supply): void <<throws IllegalArgumentException>>
FamilyRelation
- personOne: DisasterVictim
- personTwo: DisasterVictim
- relationshipTo: String
+ FamilyRelation(personOne: DisasterVictim, relationshipTo: String, personTwo: DisasterVictim) <<throws IllegalArgumentException>>
+ getPersonOne(): DisasterVictim
+ setPersonOne(personOne: DisasterVictim): void <<throws IllegalArgumentException>>
+ getPersonTwo(): DisasterVictim
+ setPersonTwo(personTwo: DisasterVictim): void <<throws IllegalArgumentException>>
+ getRelationshipTo(): String
+setRelationshipTo(relationshipTo: String): void
Supply
- type: String
- supplyId: int
- allocatedId: Integer
- quantity: int
- perishable: boolean
- expirationDate: LocalDate
- PERISHABLE_TYPES: string[] (static, final)
+ Supply (type: String, quantity: int) <<throws IllegalArgumentException>>
+ getType(): String
+ setType(type: String): void
+ getSupplyId(): int
+ setSupplyId(supplyId: int) : void
+ getAllocatedId(): Integer
+ setAllocatedId(allocatedVictimId: Integer): void
+ getQuantity(): int
+ setQuantity(quantity: int) : void <<throws IllegalArgumentException>>
+ isPerishable(): boolean
+ getExpirationDate(): LocalDate
+ setExpirationDate(expiryDate : LocalDate): void <<throws IllegalArgumentException>>
+ isExpired() : boolean
- isPerishableType(type: String): boolean
MedicalRecord
- MedicalRecordId: int
- location: Location
- treatmentDetails: String
- dateOfTreatment: LocalDate
+ MedicalRecord(location: Location, treatmentDetails: String, dateOfTreatment: LocalDate) <<throws IllegalArgumentException>>
+ getLocation(): Location
+ setLocation(location: Location): void
+ getMedicalRecordId(): int
+ setMedicalRecordId(recordId: int): void
+ getTreatmentDetails(): String
+ setTreatmentDetails(treatmentDetails: String): void
+ getDateOfTreatment(): LocalDate
+ setDateOfTreatment(dateOfTreatment: LocalDate): void <<throws IllegalArgumentException>>
Inquirer
- FIRST_NAME: String (final)
- LAST_NAME: Sting (final)
- SERVICES_PHONE: String (final)
- INFO: String (final)
+ Inquirer (firstName: String, lastName: String, phone: String, info: String)
+ getFirstName(): String
+ getLastName(): String
+ getServicesPhoneNum(): String
+ getInfo(): String
DisasterVictim
- firstName: String
- lastName:String
- personId: int
- ENTRY_DATE: LocalDate (final)
- dateOfBirth: LocalDate
- approximateAge: Integer
- gender: String
- comments: String
- familyConnections: FamilyRelation[] = new FamilyRelation[0]
- medicalRecords: MedicalRecord[] = new MedicalRecord[0]
- culturalRequirements: ArrayList<CulturalRequirement> = new ArrayList<>()
- personalBelongings: Supply[] = new Supply[0]
+ DisasterVictim(firstName: String, ENTRY_DATE: LocalDate) <<throws IllegalArgumentException>>
+ DisasterVictim(firstName: String, ENTRY_DATE: LocalDate, dateOfBirth: LocalDate)
+ getFirstName(): String
+ setFirstName(firstName: String): void
+ getLastName(): String
+ setLastName(lastName: String): void
+ getPersonId(): int
+ setPersonId(personId: int): void
+ getEntryDate(): LocalDate
+ getDateOfBirth(): LocalDate
+ setDateOfBirth(dateOfBirth: LocalDate): void <<throws IllegalArgumentException>>
+ getApproximateAge(): Integer
+ setApproximateAge(approximateAge: Integer): void
+ getGender(): String
+ setGender(gender: String) : void <<throws IllegalArgumentException>>
+ getComments(): String
+ setComments(comments: String): void
+ getFamilyConnections(): FamilyRelation[]
+ setFamilyConnections(connections: FamilyRelation[]): void
+ addFamilyConnection(record: FamilyRelation): void <<throws IllegalArgumentException>>
+ getMedicalRecords(): MedicalRecord[]
+ setMedicalRecords(records: MedicalRecord[]): void
+ addMedicalRecord(record: MedicalRecord): void
+ getPersonalBelongings(): Supply[]
+ setPersonalBelongings(belongings: Supply[]): void
+ addPersonalBelonging(supply: Supply): void
+ removePersonalBelonging(unwantedSupply: Supply): void <<throws IllegalArgumentException>>
+ getCulturalRequirements(): ArrayList<CulturalRequirement>
+ setCulturalRequirements (requirements: ArrayList<CulturalRequirement>): void
+ addCulturalRequirements (requirement: CulturalRequirement): void
+ removeCulturalRequirement(requirementType: String): void <<throws IllegalArgumentException>>
+ removeFamilyConnection(exRelation: FamilyRelation) : void <<throws IllegalArgumentException>>
personOne and personTwo
ReliefService
- inquirer: Inquirer
- dateOfInquiry: LocalDate
- missingPerson: DisasterVictim
-infoProvided: String
- lastKnownLocation: Location
+ ReliefService(inquirer: Inquirer, missingPerson: DisasterVictim, dateOfInquiry: LocalDate, infoProvided: String, lastKnownLocation: Location)
+ getInquirer(): Inquirer
+ setInquirer(inquirer: Inquirer): void
+ getDateOfInquiry(): LocalDate
+ setDateOfInquiry(dateOfInquiry: LocalDate): void <<throws IllegalArgumentException>>
+ getMissingPerson(): DisasterVictim
+ setMissingPerson(missingPerson: DisasterVictim): void
+ getInfoProvided(): String
+ setInfoProvided(infoProvided: String): void
+ getLastKnownLocation(): Location
+ setLastKnownLocation(lastKnownLocation: Location): void
+ getLogDetails(): String
Skill
- skillId: int
- disasterVictimSkillId: int
- skillName: String
- category: String
- proficiencyLevel: String
- PROFICIENCY_LVLS: String[] (static, final)
- CATEGORY_MEDICAL: String (static, final) = "medical"
- CATEGORY_LANGUAGE: String (static, final) = "language"
- CATEGORY_TRADE: String (static, final) = "trade"
+ Skill(skillName: String, category: String, proficiencyLevel: String) <<throws IllegalArgumentException>>
+ getSkillId(): int
+ setSkillId(skillId: int): void
+ getDisasterVictimSkillId(): int
+ setDisasterVictimSkillId(victimSkillId: int): void
+ getSkillName(): String
+ setSkillName(skillName: String): void
+ getCategory(): String
+ getProficiencyLevel(): String
+ setProficiencyLevel(proficiencyLevel: String): void
- isValidCategory(category: String): boolean
- isValidProficiency(proficiency: String): boolean
MedicalSkill
- certificationInfo: String
- certificationExpiration: LocalDate
- VALID_TYPES: String[] (static, final)
+ MedicalSkill(skillName: String, proficiencyLevel: String, certificationInfo: String, certificationExpirationDate: LocalDate) <<throws IllegalArgumentException>>
+ getCertificationInfo(): String
+ setCertificationInfo(certificationInfo: String): void
+ getCertificationExpirationDate(): LocalDate
+ setCertificationExpirationDate(certificationExpiry: LocalDate): void
- isValidMedicalType(skillName: String) : boolean
TradeSkill
- VALID_TYPES: String[] (static, final)
+ TradeSkill(skillName: String, proficiencyLevel: String) <<throws IllegalArgument>>
-isValidTradeType(skillName: String): boolean
LanguageSkill
- capabilities : ArrayList<String>
- CAPABILITY_READ_WRITE: String (static, final)
- CAPABILITY_SPEAK_LISTEN: String (static, final)
+ LanguageSkill(language: String, proficiencyLevel: String, capabilities: ArrayList<String>) <<throws IllegalArgumentException>>
+ getCapabilities(): ArrayList<String>
+ setCapabilities(capabilities: ArrayList<String>): void
+ getStringCapabilities(): String
-isValidCapability(capability: String): boolean
CulturalRequirement
- requirementType: String
- selectedOption: String
+ CulturalRequirement(requirementType: String, selectedOption: String)
+ getRequirementType(): String
+ setRequirementType(requirementType: String): void
+ getSelectedOption(): String
+ setSelectedOption(selectedOption: String): void
CulturalOptions
- accommodations: HashMap<String, Set<String>>
- serialVersionUID: long (static, final) = 1L
+ CulturalOptions()
+ getAccommodations(): HashMap<String, Set<String>>
+ setAccommodations(accommodations: HashMap<String, Set<String>>): void
Main
+ main(args: String[]): void
AvailableRequirementsLoader
+ load(): CulturalOptions <<static, throws IOException, ClassNotFoundException>>
Use
Extends